

DeepSkin — System Implementation Plan

For Design Handoff

1. Product Summary

DeepSkin is a **doctor-mediated** skin lesion screening tool. Patients remotely submit photos of skin lesions. An AI model (EfficientNetB2 + CBAM, binary benign/malignant classifier) analyzes each image and surfaces it to a doctor with a priority score. The doctor reviews the image, the AI's prediction, and an attention-map visualization, then makes the clinical call and communicates it to the patient.

The core design principle: patients never see the AI's prediction, confidence score, or attention map — only the doctor's final verdict. The AI is a triage/prioritization aid for doctors, not a diagnostic result for patients. This should be treated as a hard constraint throughout the design, not a soft guideline.

2. Roles & Responsibilities

2.1 Patient

Can do	Cannot do
Register / log in	See model prediction, confidence, or attention map
Upload lesion image(s) — one upload per lesion of concern	See other patients' data
Track status of each submitted case ((Pending) → (In Review) → (Reviewed))	Pick which doctor reviews their case
View doctor's final verdict + recommendation	Edit/delete a case after a doctor has picked it up
Message the doctor assigned to their case	Access admin or doctor views
View history of their own past cases	

2.2 Doctor

Can do	Cannot do
View a shared queue of unassigned cases, sorted by AI urgency (highest suspicion first)	Manage other doctors' accounts
Pick up (self-assign) a case from the queue	See cases they haven't picked up in detail (queue shows limited preview only)
View full case detail once assigned: original image(s), AI prediction, confidence, attention-map heatmap	Access system-wide analytics/audit logs
Record a structured verdict (e.g. Reassure / Recommend Biopsy / Refer to Specialist)	Trigger model retraining
Send free-text messages to the patient	
View that patient's full case history (past lesions, past verdicts)	

2.3 Admin

Can do	Cannot do
Create/deactivate doctor accounts	(Full system access — no meaningful restriction)
View system-wide dashboards: queue length, average time-to-review, case volume	
View full audit log (every case, every doctor, every verdict)	
Trigger/monitor model retraining pipeline	
Full access to all patient cases (oversight only, not routine use)	

3. Core Workflow (End to End)

1. Patient registers/logs in
 2. Patient uploads a photo of a lesion + optional note ("itchy for 2 weeks")
→ this creates a "Case"
 3. Backend runs the image through the preprocessing pipeline (hair removal, resize) then the model → produces prediction + confidence + attention map
 4. Case enters the shared Doctor Queue, sorted by model confidence (highest suspicion surfaces first – NOT shown to patient)
 5. Any available doctor opens the queue and picks up a case
→ case becomes "assigned," disappears from other doctors' queues
 6. Doctor opens case detail: sees original image, AI prediction/confidence, attention-map heatmap, patient's note, patient's past case history
 7. Doctor records a structured verdict:
 - Reassure (benign, no action needed)
 - Monitor (benign but recheck in X weeks)
 - Recommend Biopsy
 - Refer to Specialist
 8. Doctor sends a message to the patient (freeform) alongside the verdict
 9. Patient is notified, sees the verdict + doctor's message (never sees the AI's number/heatmap)
 10. Case is marked "Reviewed" and logged for audit

Case Status Lifecycle



4. Queue Sorting Logic (Doctor's View)

The unassigned case queue is sorted by **AI-estimated urgency**, highest first. This is the only place model output influences anything before a doctor reviews it — and even then, doctors only see a coarse priority indicator in the queue list (not the exact confidence number), reserving full detail for the case-detail screen.

Queue list view shows per case:

- Patient's uploaded image (thumbnail)
- Time since upload
- Priority badge: [High] / [Medium] / [Low] ← derived from model confidence bucket
- "Pick up case" button

Full confidence % and attention map are ONLY shown after a doctor opens the case detail screen (post-assignment).

This two-tier reveal (bucket in queue, full detail after pickup) avoids anchoring doctors too strongly on a raw number before they've looked at the image themselves, while still using the model to prioritize triage order.

5. Screens to Design

Patient-Facing

1. **Register / Login**
2. **Dashboard** — list of the patient's cases with status badges
3. **New Case Upload** — image upload (allow retake/re-upload before submit), optional text note
4. **Case Detail (Patient View)** — uploaded image, status, doctor's verdict (once available), message thread with doctor
5. **Case History** — past reviewed cases

Doctor-Facing

6. **Login** (shared login screen, role-routed after auth)
7. **Case Queue** — sortable/filterable list of unassigned cases (priority badge, thumbnail, time waiting)
8. **Case Detail (Doctor View)** — original image, AI prediction + confidence, attention-map overlay toggle, patient's note, patient's case history, verdict form, message composer
9. **My Active Cases** — cases this doctor has picked up but not yet finalized
10. **Patient History Panel** — reusable component showing a given patient's past cases (embedded in Case Detail)

Admin-Facing

11. **Admin Dashboard** — queue length, avg. time-to-review, case volume over time
 12. **Doctor Management** — list/add/deactivate doctor accounts
 13. **Audit Log** — searchable/filterable table of all cases + verdicts + timestamps
-

6. Key UX Principles for the Designer

- **Patient screens should read like a telehealth app**, not a diagnostic tool — calm, reassuring tone, no medical jargon, no numbers or scary visuals. Status language like "A doctor is reviewing your photo" rather than anything AI-related.
 - **Doctor screens can and should surface the AI output prominently** — confidence score, attention-map heatmap toggle over the original image, and priority badge in the queue. This is the one place where being explicit about the AI's role is a feature, not a risk.
 - **The attention-map heatmap** should be an overlay toggle on the original image (on/off or opacity slider) — doctor should be able to compare "raw image" vs "AI-highlighted region" side by side or overlaid.
 - **Verdict form should be structured, not just a text box** — dropdown/radio for the 4 verdict categories, with an optional free-text field for clinical notes. This keeps the audit trail clean and queryable.
 - **Messaging is simple async messaging** (like SMS/email thread), not real-time chat — no "typing..." indicators or read receipts needed. Refresh-to-see-new-messages is fine for this use case.
 - **Mobile-first for the patient side** (many patients will use phones to photograph lesions and check status); **desktop-first for the doctor side** (doctors reviewing detailed images benefit from a larger screen).
-

7. Technical Notes (Context for Designer, Not for Implementation)

- Frontend: React (web), mobile-responsive for patient-facing screens
 - Backend: Django REST Framework
 - Auth: JWT-based, role field on User model ((`patient`)) / ((`doctor`)) / ((`admin`))
 - Image storage: uploaded originals + generated attention-map overlays both need storage — design should account for image loading states (upload progress, AI processing spinner before a case appears in the doctor queue)
 - One upload = one "Case" = one lesion. Patients with multiple concerns create multiple separate cases (not one case with multiple unrelated lesion photos).
-

8. Explicitly Out of Scope (for this phase)

- Real-time chat / video calls
- Payment processing
- Multi-clinic / multi-tenant support
- Production-grade PII encryption/compliance (HIPAA-equivalent) — noted as a documented limitation, not implemented, for this project's scope
- Doctor specialty routing / tiered review (senior vs junior doctor)